

MODERN COMPUTING for PHYSICS

Neural Network Evaluation

MY CPU – MY GPU

CPU

```
Model name: Intel(R) Core(TM) i7-10710U CPU @ 1.10GHz
CPU family: 6
Model: 166
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
```

```
Caches (sum of all):
L1d: 192 KiB (6 instances)
L1i: 192 KiB (6 instances)
L2: 1.5 MiB (6 instances)
L3: 12 MiB (1 instance)
```

GPU

```
Device name: NVIDIA GeForce GTX 1650 with Max-Q Design
n SMPs: 16
n SPs: 2048
Clock rate (MHz): 1215.8
L2 Cache Size (KB): 1048.6
Memory Clock Rate (MHz): 3418
Memory Bus Width (bits): 128
Peak Memory Bandwidth (GB/s): 112.0
Total global memory (GB) 4.0
Shared memory per block (KB) 48.0
```

THE NEURAL NETWORK

```
class SimpleMLP(nn.Module):
    def __init__(self):
        super(SimpleMLP, self).__init__()
        # Flatten: 28x28 = 784
        self.fc1 = nn.Linear(784, 256)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(256, 128)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(128, 10)

    def forward(self, x):
        x = torch.flatten(x, 1) #(batch, 784)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x # return the logits
```

It's a 3-layer MLP
(2 hidden + 1 visible)

```
trainset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Trained on the MNIST handwritten digits dataset

```
arr = np.ascontiguousarray(arr)
with open(filename, 'wb') as f:
    f.write(arr.tobytes())
print(f"    Saved: {filename} (Shape: {arr.shape})")
```

It exports the produced data into a float32 NumPy array and saves it as a raw binary file.

Mind the structure of the output weights: (0, 1)

```
labels_arr = predictions.detach().cpu().numpy().astype(np.int32)
with open(os.path.join(export_dir, "test_predictions.bin"), 'wb') as f:
    f.write(labels_arr.tobytes())
print(f"    Saved: test_predictions.bin (Shape: {labels_arr.shape}, Type: int32)")
```

SERIAL IMPLEMENTATION – the math

functions

```
// X: [B, I], W: [O, I], b: [O] -> Y: [B, O]
// V0: the simplest possible implementation
std::vector<float> linear_layer_v0(\
const std::vector<float>& X, const std::vector<float>& W,\
const std::vector<float>& b, int B, int I, int O) {

    std::vector<float> Y(B * O, 0.0f); // RAM Initialization
    for (int i = 0; i < B; ++i) {
        for (int j = 0; j < O; ++j) {
            // continuous reading and WRITING in memory for each
            // single calculation --> thus naive
            for (int k = 0; k < I; ++k) {
                Y[i * O + j] = Y[i * O + j] + X[i * I + k] * W[j * I + k]; 1, 3
            }
            // addition of bias with further memory access
            Y[i * O + j] = Y[i * O + j] + b[j]; 2
        }
    }
    return Y;
}

// In-place ReLU activation function:  $f(x) = \max(0, x)$ 
void relu_inplace(std::vector<float>& X) {
    for (size_t i = 0; i < X.size(); ++i) {
        if (X[i] < 0.0f) X[i] = 0.0f;
    }
}
```

main

```
// Layer 1
auto start_l1 = std::chrono::high_resolution_clock::now();
auto H1 = linear_layer_v0(X, W1, b1, B, I1, O1);
relu_inplace(H1); 4
auto end_l1 = std::chrono::high_resolution_clock::now();

// Layer 2
auto start_l2 = std::chrono::high_resolution_clock::now();
auto H2 = linear_layer_v0(H1, W2, b2, B, I2, O2);
relu_inplace(H2); 4
auto end_l2 = std::chrono::high_resolution_clock::now();

// Layer 3 (Logits)
auto start_l3 = std::chrono::high_resolution_clock::now();
auto Logits = linear_layer_v0(H2, W3, b3, B, I3, O3);
auto end_l3 = std::chrono::high_resolution_clock::now();
```

timing

Legend:

- Improvable
- - - May be done better
- n «mistake» identification

SERIAL IMPLEMENTATION – upgrades: 1+2

```
// V1: Accumulator in Register (<-> minimizing cache writes)
// V2: Bias "trick" (Arithmetic Instruction Optimization)
std::vector<float> linear_layer_v1() {
    std::vector<float> Y(B * O, 0.0f);
    for (int i = 0; i < B; ++i) {
        for (int j = 0; j < O; ++j) {

            float sum = b[j];
            // 1) the partial sum lives in a fast register.
            // 2) initialized the register directly with the bias.

            for (int k = 0; k < I; ++k) {
                sum += X[i * I + k] * W[j * I + k];
                // --> no RAM/Cache writes here
            }

            Y[i * O + j] = sum;
            // 1) --> a single write per neuron
            // 2) --> removed the final + b[j]

        }
    }
    return Y;
}
```

$Y[i * O + j] = Y[i * O + j] + X[i * I + k] * W[j * I + k];$



To generate a single output element, I reads and I writes on Y are needed $\rightarrow 2 \times B \times O \times I$ memory accesses in total

The 'sum' variable lives on register $\rightarrow$ writes to Y drop to $B \times O$



$(2I - 1) \times B \times O$ accesses saved $\rightarrow$ arithmetic intensity increases

With the initialization of 'sum' to $b[j]$:

$B \times O$ FLOPs and $B \times O$ initializations to zero removed

SERIAL IMPLEMENTATION – upgrades: 3+4

```
// V3: Pointer arithmetic trick
// V4: Applying the relu on-the-fly
std::vector<float> linear_layer_v3() {
    std::vector<float> Y(B * O, 0.0f);
    for (int i = 0; i < B; ++i) {

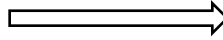
        // 3) compute the starting address of the i-th row of X
        const float* x_row = &X[i * I];

        for (int j = 0; j < O; ++j) {

            // 3) compute the starting address of the j-th row of W
            const float* w_row = &W[j * I];

            float sum = b[j];
            for (int k = 0; k < I; ++k) {
                sum += x_row[k] * w_row[k];
                // 3) --> only FMA float ops, zero index multiplications
            }

            // 4) applying the ReLU on-the-fly
            if (apply_relu) { // (this branch is perfectly predictable)
                Y[i * O + j] = std::max(0.0f, sum);
            } else {
                Y[i * O + j] = sum;
            }
        }
    }
    return Y;
}
```



```
// Layer 1
auto start_l1 = std::chrono::high_resolution_clock::now();
auto H1 = linear_layer_v4(X, W1, b1, B, I1, O1, true);
auto end_l1 = std::chrono::high_resolution_clock::now();

// Layer 2
auto start_l2 = std::chrono::high_resolution_clock::now();
auto H2 = linear_layer_v4(H1, W2, b2, B, I2, O2, true);
auto end_l2 = std::chrono::high_resolution_clock::now();

// Layer 3
auto start_l3 = std::chrono::high_resolution_clock::now();
auto Logits = linear_layer_v4(H2, W3, b3, B, I3, O3, false);
auto end_l3 = std::chrono::high_resolution_clock::now();
```

- 'v3' got rid of $2 \times B \times O \times I - (B + B \times O)$ integer multiplications in total
- 'v4' got rid of $B \times O$ memory reads and up to $B \times O$ memory writes

SERIAL IMPLEMENTATION – metrics

Results Validation

```
auto predictions = argmax(Logits, B, O3);

int correct_preds = 0;
float max_logit_diff = 0.0f;
for (int i = 0; i < B; ++i) {
    if (predictions[i] == expected_preds[i]) correct_preds++;
    for (int j = 0; j < O3; ++j) {
        float diff = std::abs(Logits[i * O3 + j] - expected_logits[i * O3 + j]);
        if (diff > max_logit_diff) max_logit_diff = diff;
    }
}

std::cout << "Accuracy vs Python: " << (correct_preds*100.0/B) << std::endl;
std::cout << "Maximum Logits Error: " << max_logit_diff << " (should be < 1e-5)";
```

```
(LCP) piewsl@MSI:~/MCP/project_NN$ g++ -O0 -std=c++17 serial_v4_relu.cpp
(LCP) piewsl@MSI:~/MCP/project_NN$ ./a.out 1000 v4_relu no_opt 1
-> Loading weights and inputs into memory...
-> Starting Forward Pass (Inference)...

-> Results Validation:
    Accuracy vs Python: 1000/1000 (100%)
    Maximum Logits Error: 3.814697e-06 (should be < 1e-5)

-> Time Breakdown per Layer:
    Layer 1 (FC + ReLU) : 528.2463 ms
    Layer 2 (FC + ReLU) : 77.4639 ms
    Layer 3 (Only FC)   : 3.0306 ms

-> HPC Figures of Merit (Batch Size = 1000):
    Inference Time      : 608.740786 ms
    Total FLOPs         : 469.888000 MFLOPs
    Total Memory I/O    : 7.188584 MB
    -----
    Performance (GFLOPS): 0.77 GFLOPS
    Bandwidth (GB/s)    : 0.01 GB/s
    CGMA (FLOPs/Byte)   : 65.37 (Arithmetic Intensity)
    -----
```

SERIAL IMPLEMENTATION – metrics

Hardware metrics

```
double flops_l1 = B * O1 * 2.0 * I1 + (B * O1);
double flops_l2 = B * O2 * 2.0 * I2 + (B * O2);
double flops_l3 = B * O3 * 2.0 * I3;
double total_flops = flops_l1 + flops_l2 + flops_l3;
```

```
double bytes_l1 = (B*I1 + O1*I1 + O1 + B*O1) * 4.0;
double bytes_l2 = (B*I2 + O2*I2 + O2 + B*O2) * 4.0;
double bytes_l3 = (B*I3 + O3*I3 + O3 + B*O3) * 4.0;
double total_bytes = bytes_l1 + bytes_l2 + bytes_l3;
```

```
double time_sec = time_l1 + time_l2 + time_l3;
double gflops = (total_flops / time_sec) / 1e9;
double bandwidth_gbps = (total_bytes / time_sec) / 1e9;
double cgma = total_flops / total_bytes; //CGMA
```

Intel Math Kernel Library (MKL) → ≈ 175 GFLOPs

STREAM benchmark → 11.1379 GB/s

(gcc -O3 -fopenmp -DSTREAM_TYPE=float -
DSTREAM_ARRAY_SIZE=20000000 STREAM.c)

Flops:
$$\underbrace{B \times O_1 \times I_1}_{\text{tot iterations}} \times 2.0 + \underbrace{(B \times O_1)}_{\text{ReLU}}$$

product&addition(&bias)

Correct estimate
for all the versions

Bytes:
$$\underbrace{(B \times I_1)}_X + \underbrace{O_1 \times I_1}_W + \underbrace{O_1}_b + \underbrace{(B \times O_1)}_{Y/H} * 4$$

float conversion

- **v0**: omits the massive Read-Modify-Write traffic on Y inside the inner loop, as well as memory pass by ReLu;
- **v1-3**: the register eliminates redundant traffic on Y , but still omitting ReLu memory transfers;
- **v4**: fixes the extra transfers due to the ReLu.

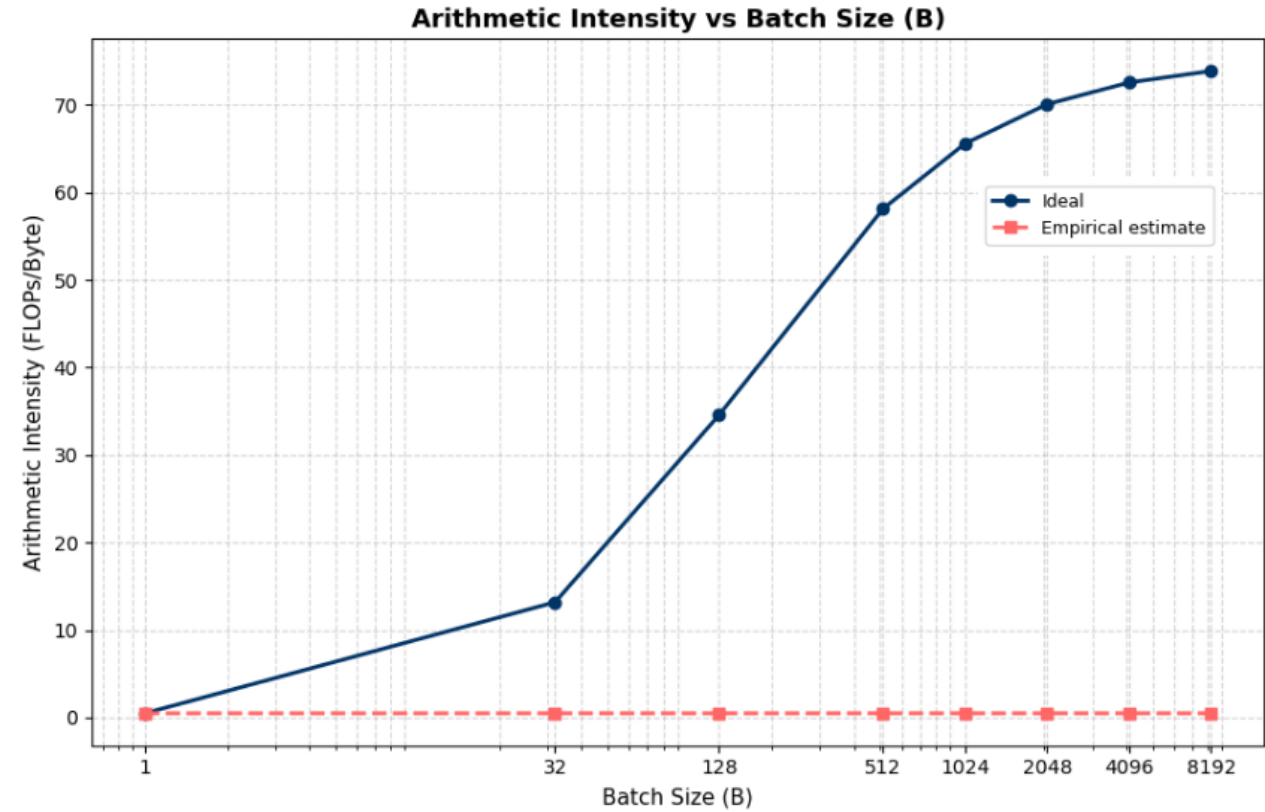
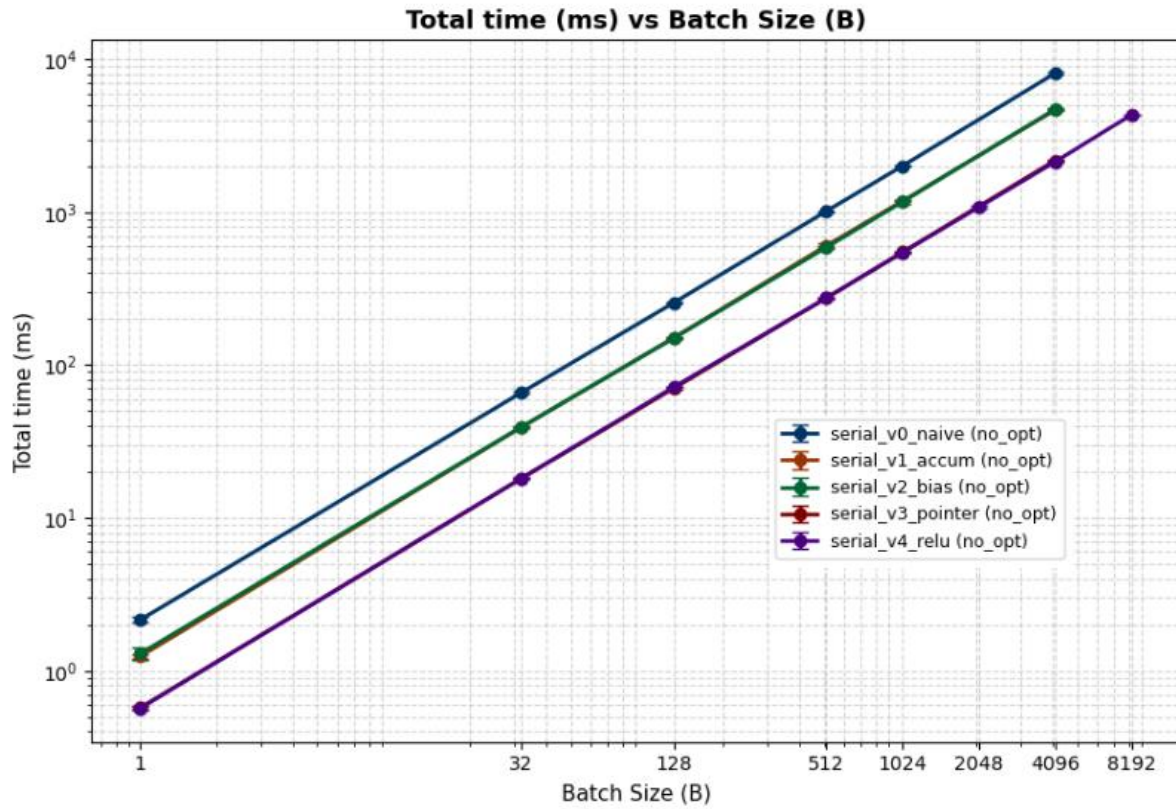
Note: $W1 \approx 803$ KiB → can only fit in the L3 cache → per every i in B , $W1$ will be entirely re-read, so:

$B * (O_1 \times I_1)$ provides a more accurate contribution to 'bytes_l1'.

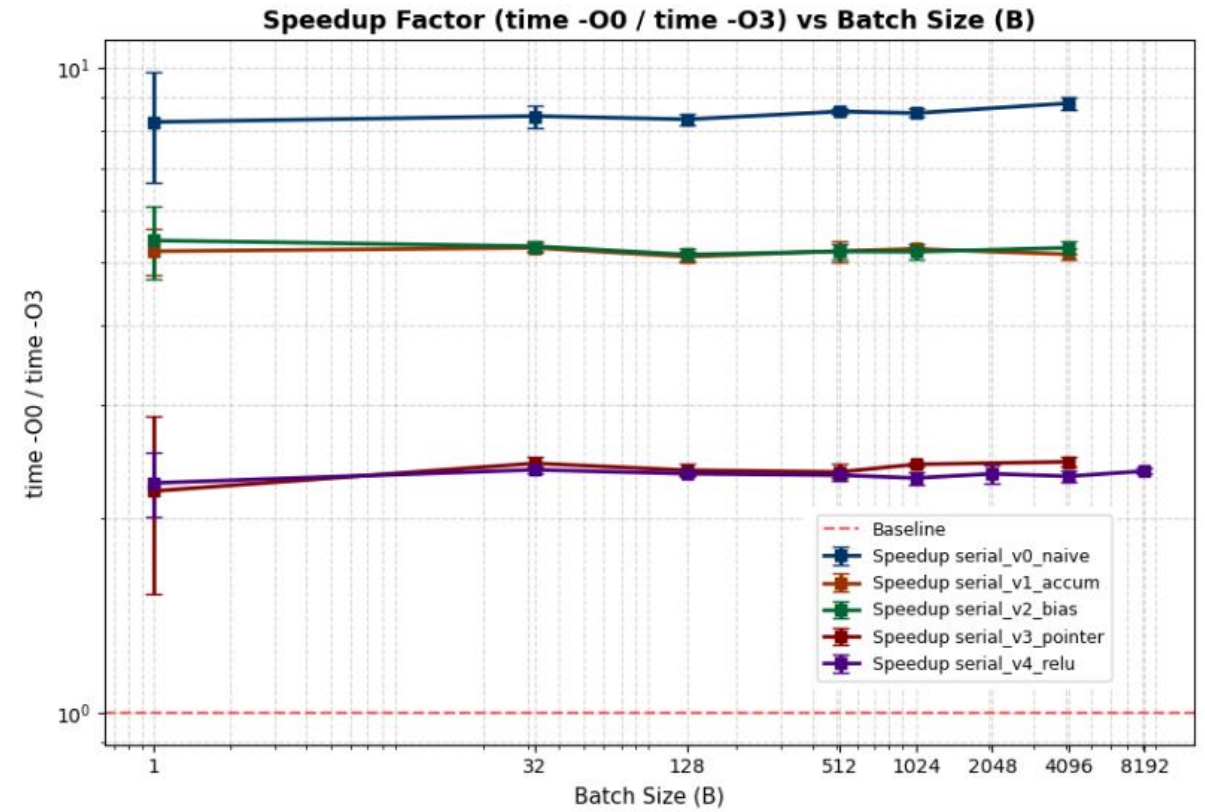
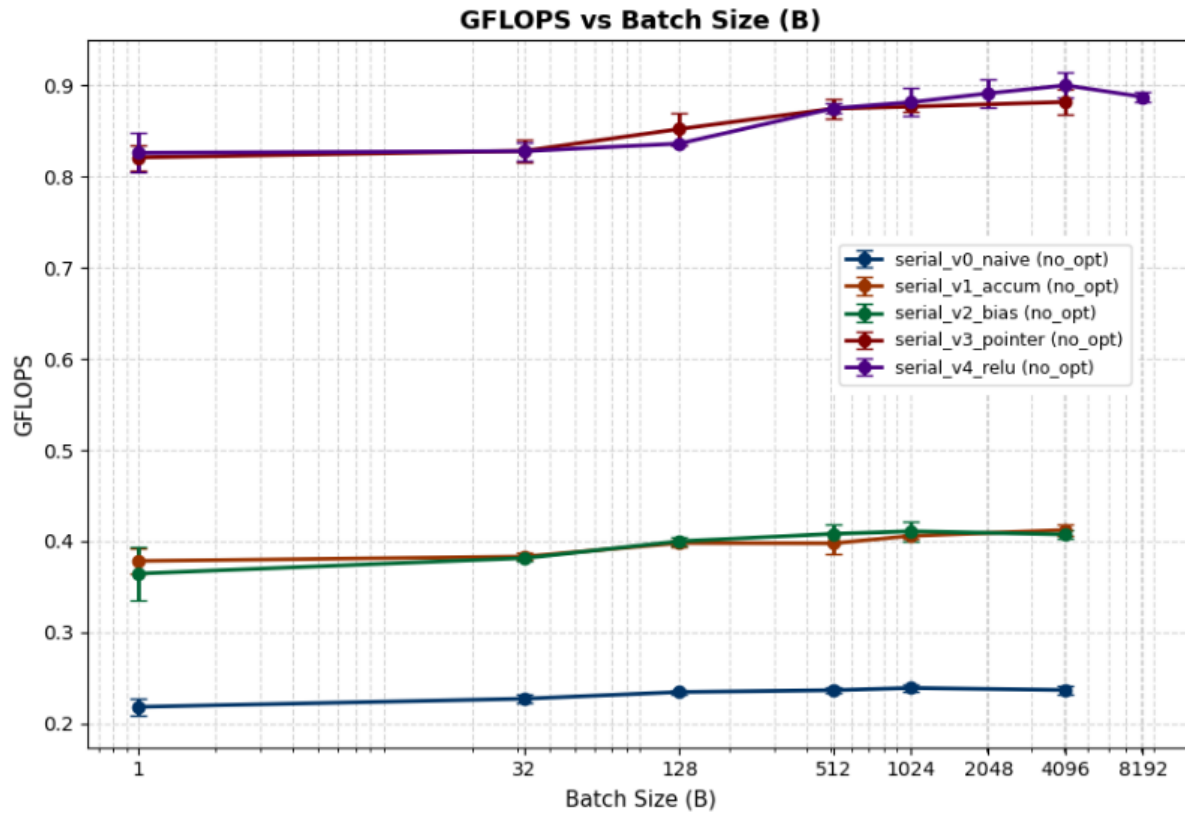
$W2 \approx 131$ KiB → fits in the L2

$W3 \approx 5$ KiB → fits in the L1d

SERIAL IMPLEMENTATION – results



SERIAL IMPLEMENTATION – results



OpenMP – versions

‘omp_v2/v4’: applied ‘#pragma omp parallel for’
to the serial v2/v4

```
// V4: Parallelizing 'serial_v4'
std::vector<float> linear_layer_v4() {
    std::vector<float> Y(B * O, 0.0f);

    #pragma omp parallel for default(none) \
        shared(X, W, b, Y, B, I, O, apply_relu) \
        collapse(2) schedule(static)

    for (int i = 0; i < B; ++i) {
        for (int j = 0; j < O; ++j) {

            const float *x_row = &X[i * I];
            const float *w_row = &W[j * I];
            float sum = b[j];

            for (int k = 0; k < I; ++k) {
                sum += x_row[k] * w_row[k];
            }

            if (apply_relu) {
                Y[i * O + j] = std::max(0.0f, sum);
            } else {
                Y[i * O + j] = sum;
            }
        }
    }
    return Y;
}
```

- The variables flagged as 'shared' are safe: read-only input and output, or independent writes.
- ‘collapse(2)’ creates B*O independent tasks.
- ‘schedule(static)’ because the workload (I iterations) is identical for each task.
- ‘x_row, w_row, i, j, k’ are implicitly private because they are declared and initialized locally
- The write to Y is safe without atomic because the index (i * O + j) is unique for the task

OpenMP – versions

‘omp_v3’: applied ‘#pragma omp parallel for’ to the serial v3

```
// V3: Parallelization with collapse(1) of 'serial_v3'
std::vector<float> linear_layer_v3() {
    std::vector<float> Y(B * O, 0.0f);
    #pragma omp parallel for default(none) \
        shared(X, W, b, Y, B, I, O) \
        collapse(1) schedule(static)

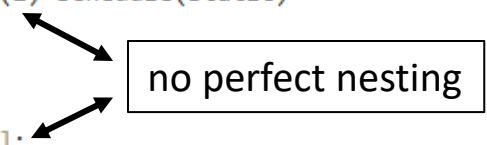
    for (int i = 0; i < B; ++i) {

        const float *x_row = &X[i * I];
        // Keeping the micro-optimization sacrificing 'collapse(2)'

        for (int j = 0; j < O; ++j) {

            const float *w_row = &W[j * I];

            float sum = b[j];
            for (int k = 0; k < I; ++k) {
                sum += x_row[k] * w_row[k];
            }
            Y[i * O + j] = sum;
        }
    }
    return Y;
}
```



no perfect nesting

‘omp_v2/v3_relu’: as ‘omp_v2/v3’ but parallelizing also the ReLu function

```
void relu_inplace(std::vector<float>& X) {
    size_t n = X.size();
    // to avoid method calls (X.size()) inside the for

    #pragma omp parallel for default(none) \
        shared(X, n) \
        schedule(static)

    for (int i = 0; i < n; ++i) {
        if (X[i] < 0.0f) X[i] = 0.0f;
    }
}
```

OpenMP – versions

'omp_v4_singlefunct, omp_v5': apply the 'omp_v3/v4' idea but implement the forward pass of all 3 layers inside a single function.

```
// V5: Full forward pass of a 3-layer network inside a SINGLE parallel region
// <-> attempt to avoid multiple fork-join overheads by reusing the same
// thread pool, USING collapse(1), to restore the pointer arithmetic trick
std::vector<float> mlp_forward_unified_v2(
    const std::vector<float>& X,
    const std::vector<float>& W1, const std::vector<float>& b1,
    const std::vector<float>& W2, const std::vector<float>& b2,
    const std::vector<float>& W3, const std::vector<float>& b3,
    int B,
    double& time_l1, double& time_l2, double& time_l3, double& time_sec
) {
    const int I1 = 784, O1 = 256;
    const int I2 = 256, O2 = 128;
    const int I3 = 128, O3 = 10;

    std::vector<float> H1(B * O1, 0.0f);
    std::vector<float> H2(B * O2, 0.0f);
    std::vector<float> Logits(B * O3, 0.0f);

    double t_l1_end = 0.0, t_l2_end = 0.0;
    double t_start = omp_get_wtime(); // START TIMING
    #pragma omp parallel default(none) \
        shared(X, W1, b1, H1, W2, b2, H2, W3, b3, Logits, B, \
            t_l1_end, t_l2_end)
    { // Start of the parallel region
```

```
        int tid = omp_get_thread_num();

        // LAYER 1
        #pragma omp for collapse(1) nowait
        // 'nowait' removes the implicit barrier at the end of the loop
        for (int i = 0; i < B; ++i) {
            const float *x_row = &X[i * I1];
            for (int j = 0; j < O1; ++j) {
                const float *w_row = &W1[j * I1];
                float sum = b1[j];
                for (int k = 0; k < I1; ++k) sum += x_row[k] * w_row[k];
                H1[i * O1 + j] = std::max(0.0f, sum);
            }
        }
        // Explicit barrier:
        #pragma omp barrier

        if (tid == 0) t_l1_end = omp_get_wtime(); // Layer 1 time

        // LAYER 2
        // ...
        // LAYER 3
        // ...

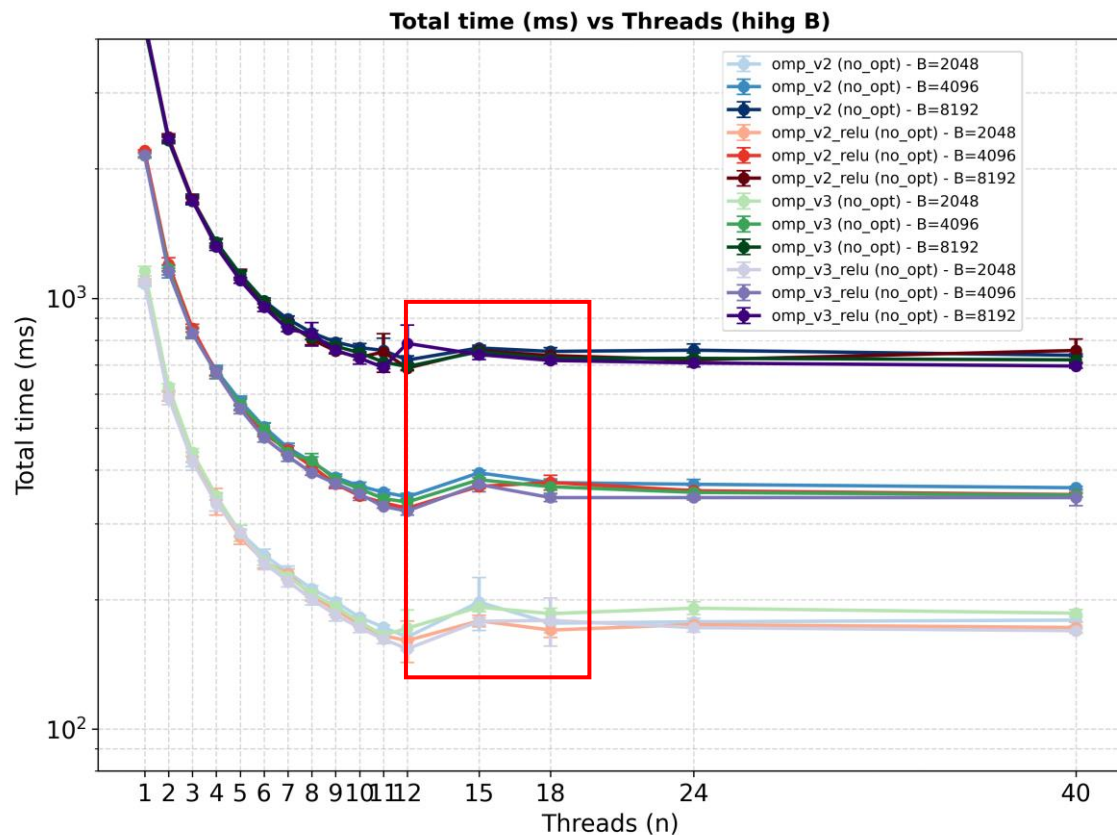
    } // End of the parallel region
    double t_end = omp_get_wtime(); // END TIMING

    // Durations
    time_l1 = t_l1_end - t_start;
    time_l2 = t_l2_end - t_l1_end;
    time_l3 = t_end - t_l2_end;
    time_sec = t_end - t_start;

    return Logits;
}
```

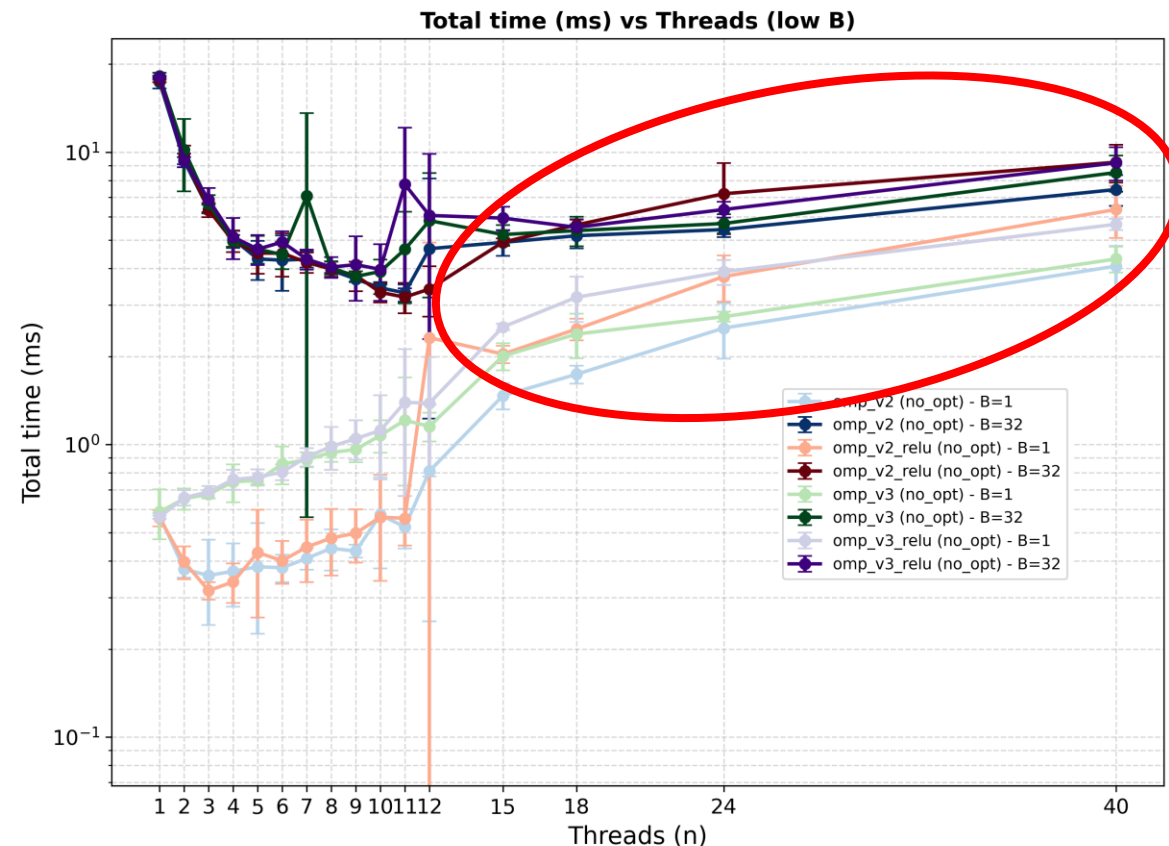

OpenMP – results: v2/v2_relu, v3/v3_relu

$B=[2048, 4096, 8192]$



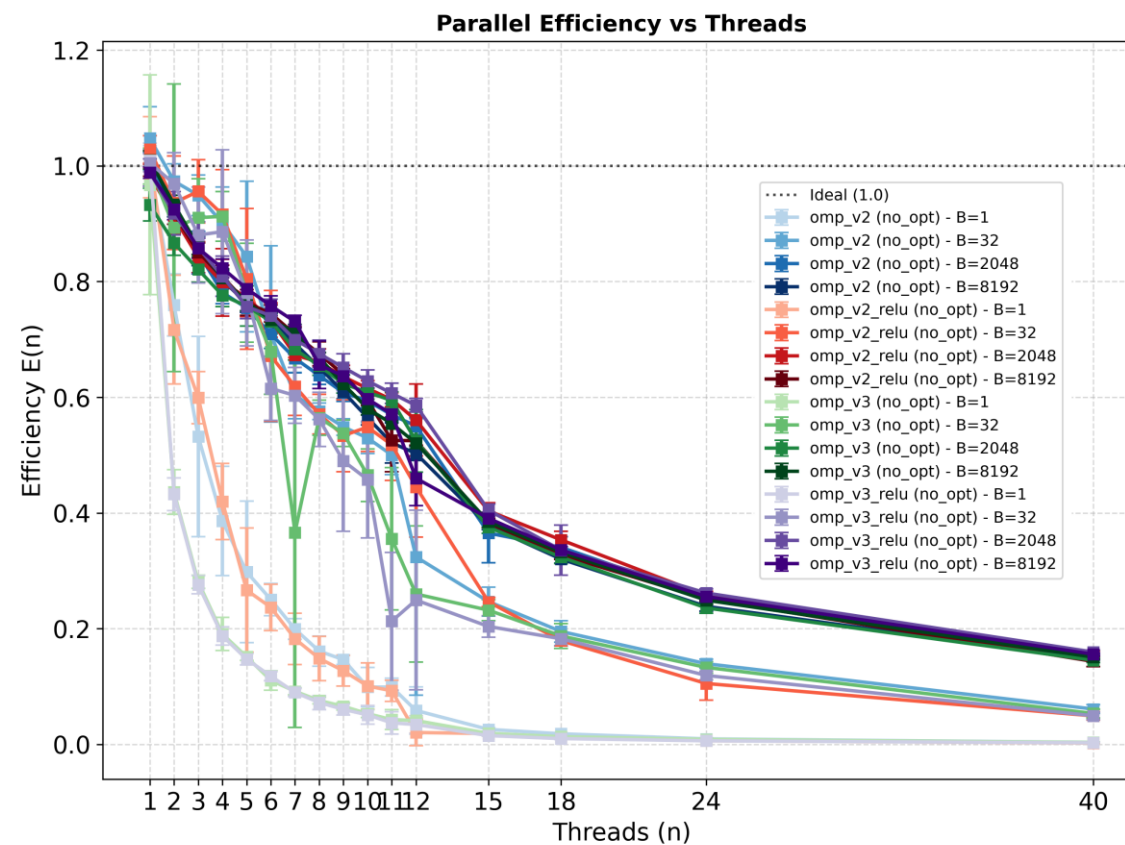
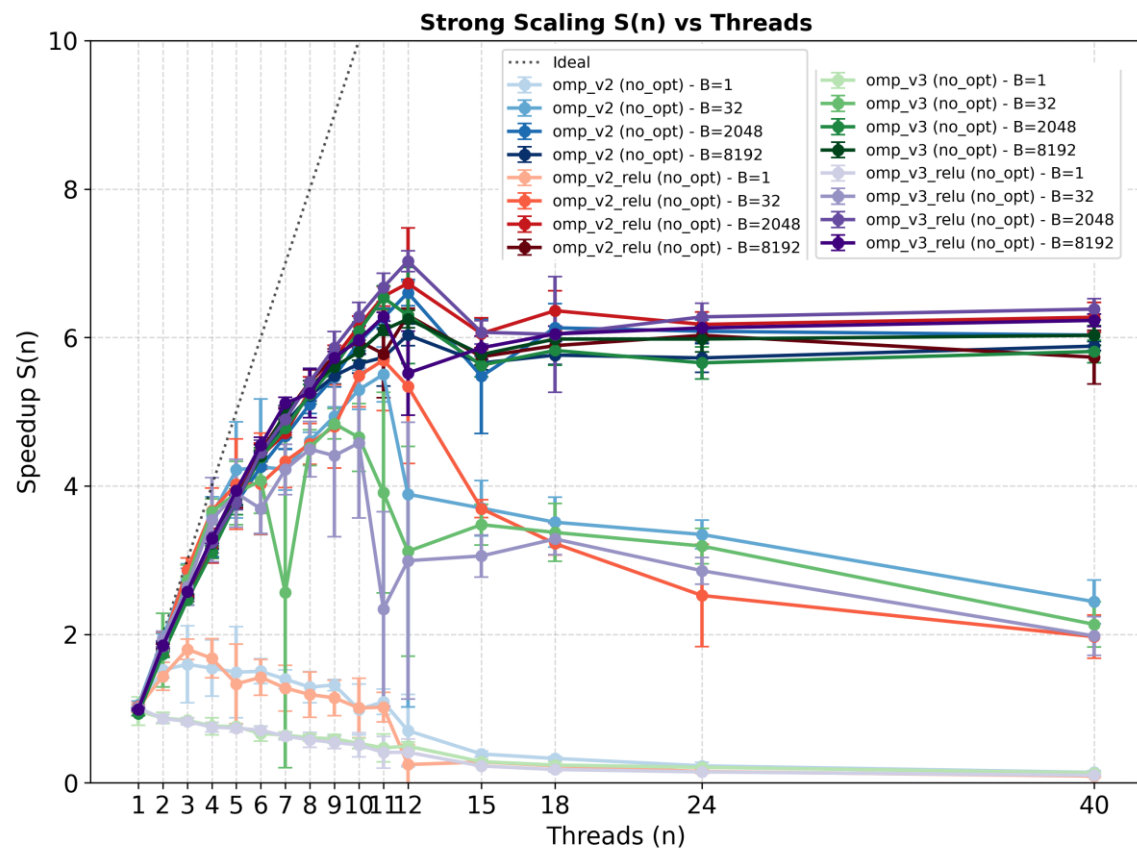
Sostanziale compatibilità entro 1σ , ma v3 è sistematicamente migliore di v2. Anche le versioni _relu mantengono quest'ordine e mostrano compatibilità con le relative versioni seriali.

$B=[1, 32]$



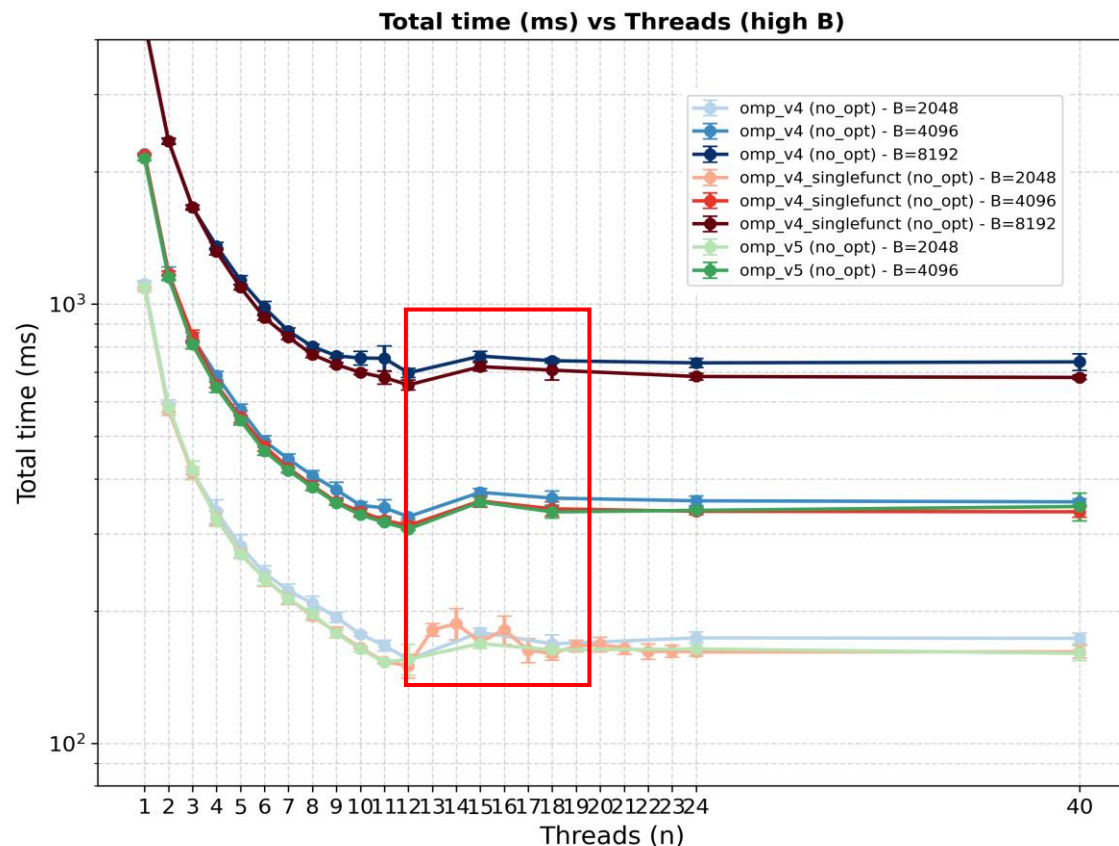
Parallelizzare la ReLU esplicitamente può essere dannoso: $v3 > v3_relu$ (specialmente per $n > 12$ se $B = 1$, $n > 18$ se $B = 32$). Lo stesso vale per v2 vs v2_relu. v3_relu è drammaticamente la peggiore per $B = 1$.

OpenMP – results: v2/v2_relu, v3/v3_relu



OpenMP – results: v4/v4_singlefunct, v5

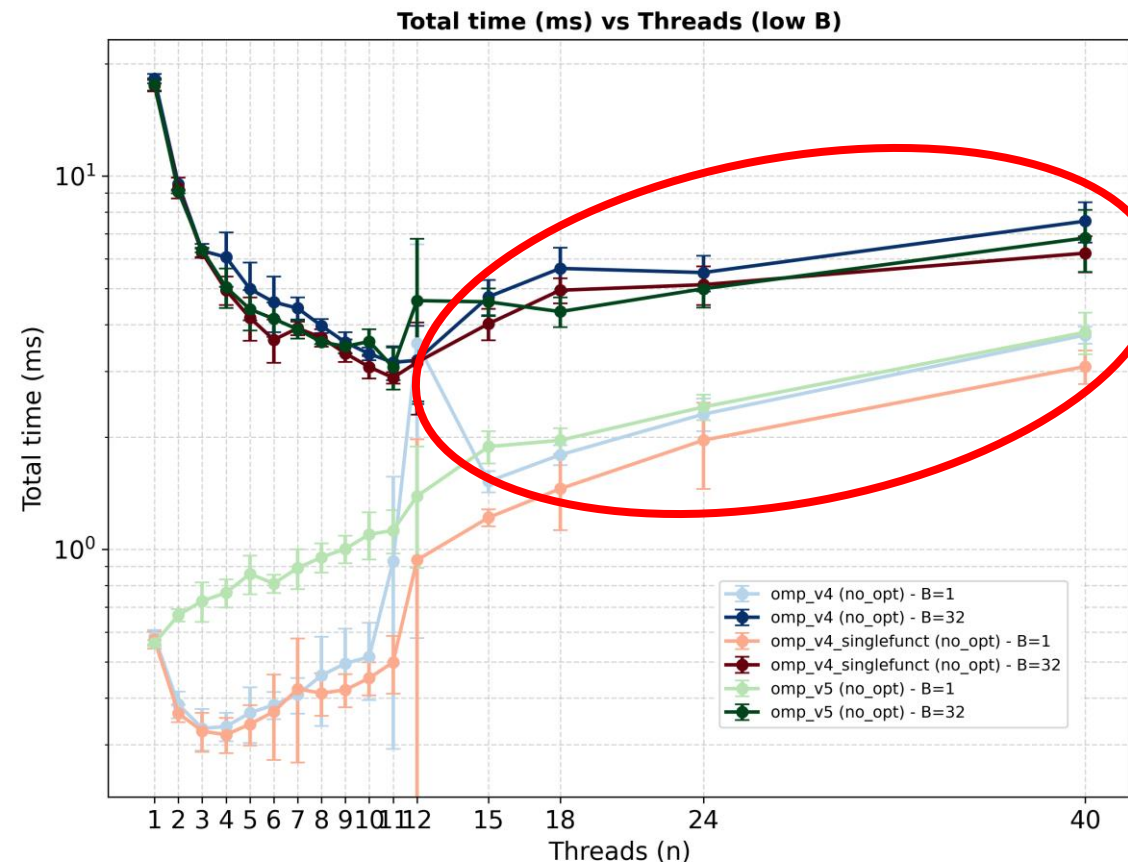
$B=[2048, 4096, 8192]$



v4_singlefunct è sistematicamente migliore di v4, anche se spesso compatibile entro 1σ ; ci sono eccezioni puntuali di compatibilità a 2σ per specifici $n \geq 8$.

v4_singlefunct e v5: identità di performance.

$B=[1, 32]$



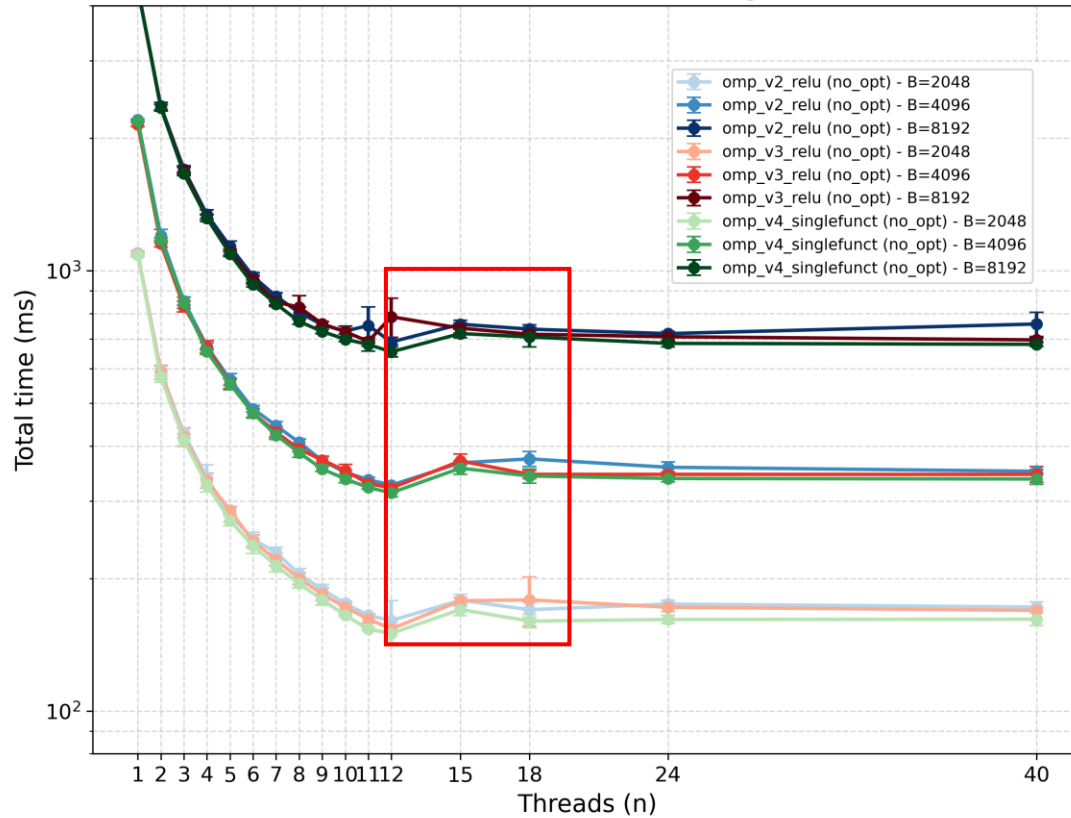
v4_singlefunct > v4 nei layer 2 e 3 (non mostrati): incompatibilità a 3σ per $n > 12$.

Per $B = 32$, v4_s e v5 sono fortemente compatibili, ma per $B = 1$, v4_s è drammaticamente migliore di v5.

OpenMP – results: v4_singlefunct, v2/3_relu

B=[2048, 4096, 8192]

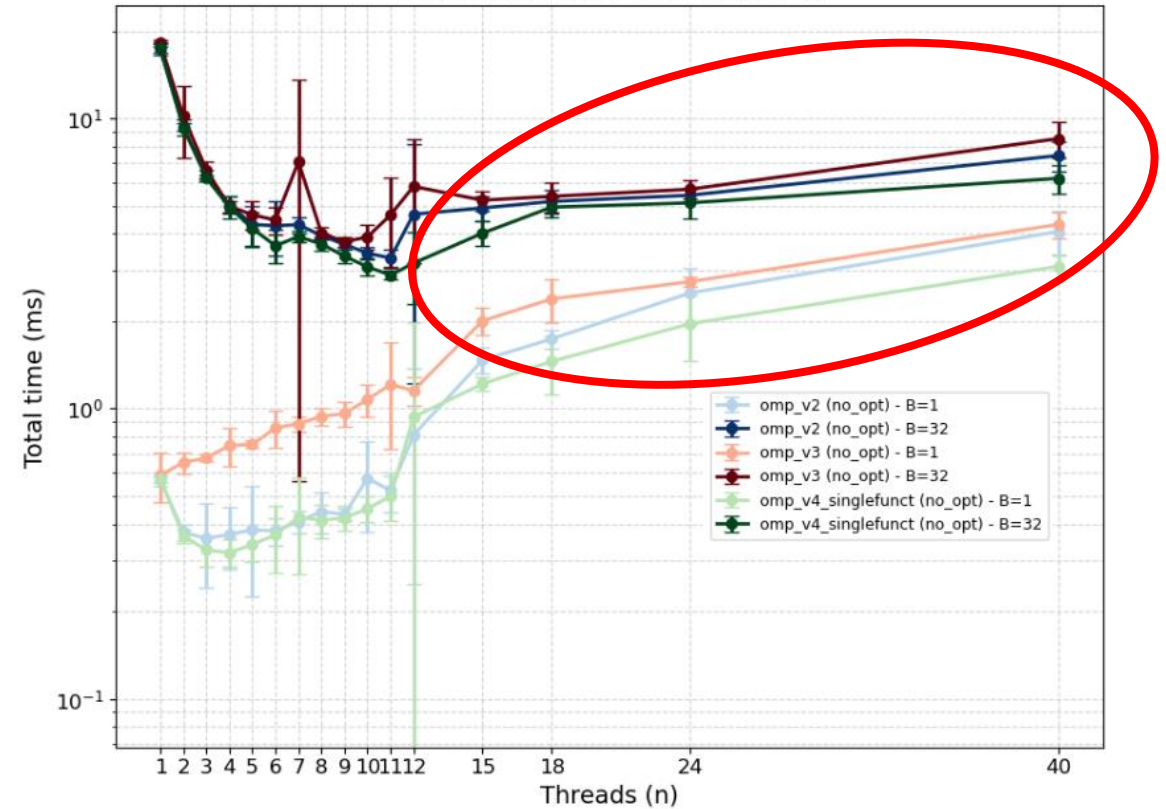
Total time (ms) vs Threads (high B)



v4_singlefunct > v3_relu > v2_relu, ma con compatibilità generalmente entro 2σ .

B=[1, 32]

Total time (ms) vs Threads (low B)



v4_singlefunct >> v2, v3, n > 12 <-> influence of the reduced overhead

OpenMP – results: summing up

ReLU parallelization:

For small batches, it becomes harmful, with v3_relu becoming *dramatically the worst* for $B = 1$.



In those plots we can actually see the influence of the overhead (the red circles)

For large B it does not significantly change performance compared to the serial version:

- Does not worsen them: expected, because the overhead is amortized
- Does not improve them: unexpected → we are witnessing the memory-boundedness of the task



Further proved by t_{tot} plateauing for $n \geq 18$

'collapse(1)':

Should prevent False Sharing that however is negligible for high enough B (32 already), and for $B=1$ the advantage gets overwhelmed by the effects of serialization

Kernel Fusion:

Visible effects, mainly for low B s and when paired with “singelfunction”

«red rectangles»:

For $12 < n \leq 15$ and high B , t_{tot} increases because the chunk size has not shrunk enough to compensate for the fact that some/the cores have to handle two of them;
For $n \geq 18$ t_{tot} plateaus, but to a value higher than $t_{\text{tot}}(12)$, due to the overhead of context switching

OpenACC – versions

‘oacc_v1’: the simplest working attempt at GPU parallelization of the serial code

// V1: the laziest possible approach to parallelizing on GPU:

```
void linear_layer_v4_step1(const float* restrict X,
                          const float* restrict W,
                          const float* restrict b,
                          float* restrict Y,
                          int B, int I, int O, bool apply_relu) {
    #pragma acc kernels
    {
        for (int i = 0; i < B; ++i) {
            for (int j = 0; j < O; ++j) {
                float sum = b[j];
                for (int k = 0; k < I; ++k) {
                    sum += X[i * I + k] * W[j * I + k];
                }

                if (apply_relu) {
                    // avoiding std::max host-device confusion
                    Y[i * O + j] = sum > 0.0f ? sum : 0.0f;
                } else {
                    Y[i * O + j] = sum;
                }
            }
        }
    }
}
```

main

// Pre-allocating CPU memory for the intermediate and final outputs

```
std::vector<float> H1(B * O1);
std::vector<float> H2(B * O2);
std::vector<float> Logits(B * O3);
```

// Extracting the raw pointers for safety when using OpenACC

```
float* ptr_X = X.data();
float* ptr_W1 = W1.data(); float* ptr_b1 = b1.data(); float* ptr_H1 = H1.data();
float* ptr_W2 = W2.data(); float* ptr_b2 = b2.data(); float* ptr_H2 = H2.data();
float* ptr_W3 = W3.data(); float* ptr_b3 = b3.data(); float* ptr_Logits = Logits.data();
```

// Layer 1

```
auto start_l1 = std::chrono::high_resolution_clock::now();
linear_layer_v4_step1(ptr_X, ptr_W1, ptr_b1, ptr_H1, B, I1, O1, true);
auto end_l1 = std::chrono::high_resolution_clock::now();
```

//...

no «data region»

- “simplest” <-> using a single 'kernels' directive and nothing else, but passing the arrays using the 'restrict' keyword.
- No data structure used, not even in the 'main'
--> must compile with the flag '-gpu=mem:managed'

OpenACC – versions

'oacc_v1':

```
(LCP) piewsl@MSI:~/MCP/project_NN$ nvc++ -acc -gpu=cc75 -Minfo=accel -gpu=mem:managed -O0 oacc_v1.cpp
linear_layer_v4_step1(float const*, float const*, float const*, float*, int, int, int, bool):
  47, Generating implicit copyin(X[:]) [if not already present]
      Generating implicit copy(Y[:]) [if not already present]
      Generating implicit copyin(b[:0],W[:]) [if not already present]
  48, Loop carried dependence of Y-> prevents parallelization
      Loop carried dependence of Y-> prevents vectorization
      Loop carried backward dependence of Y-> prevents vectorization
  49, Loop is parallelizable
      Generating NVIDIA GPU code
  48, #pragma acc loop seq
  49, #pragma acc loop gang /* blockIdx.x */
  51, #pragma acc loop vector(128) /* threadIdx.x */
      Generating implicit reduction(+:sum)
  51, Loop is parallelizable
```

'-gpu=mem:managed' is doing its job

Loop on B won't be parallelized nor optimized

Loop on O will be parallelized, across gangs(<->blockIdx.x)

Reduction operation recognized and potential race condition tackled; parallelized across threads in a block of 128

OpenACC – versions

‘oacc_v2’: forcing the parallelization of the non-parallelized loop of v1

*// V2: V1 refused to parallelize the outermost loop --> this is still the laziest,
// but more parallelized version --> still to be compiled with '-gpu=mem:managed'*

```
void linear_layer_v4_step2(const float* restrict X,  
                           const float* restrict W,  
                           const float* restrict b,  
                           float* restrict Y,  
                           int B, int I, int O, bool apply_relu) {  
    #pragma acc kernels  
    {  
        #pragma acc loop independent  
        // It is promising the compiler it is safe to parallelize this loop  
        for (int i = 0; i < B; ++i) {  
  
            for (int j = 0; j < O; ++j) {  
                float sum = b[j];  
  
                for (int k = 0; k < I; ++k) {  
                    sum += X[i * I + k] * W[j * I + k];  
                }  
  
                if (apply_relu) {  
                    Y[i * O + j] = Y[i * O + j] = sum > 0.0f ? sum : 0.0f;  
                } else {  
                    Y[i * O + j] = sum;  
                }  
            }  
        }  
    }  
}
```

```
45, Generating implicit copyin(X[:]) [if not already present]  
    Generating implicit copy(Y[:]) [if not already present]  
    Generating implicit copyin(b[:0],W[:]) [if not already present]  
48, Loop is parallelizable  
50, Loop is parallelizable  
    Generating NVIDIA GPU code  
48, #pragma acc loop gang collapse(2) /* blockIdx.x */  
50, /* blockIdx.x auto-collapsed */  
56, #pragma acc loop vector(128) /* threadIdx.x */  
    Generating implicit reduction(+:sum)  
56, Loop is parallelizable
```

BxO mapped to thread blocks

- So the mapping of the problem to the gpu hardware is pretty well done already
- But how about memory management?
- We will inspect it later

OpenACC – versions

‘oacc_v3’: optimal ‘kernels’ version

```
// V3: optimal 'kernel' based version
void linear_layer_v4_step3(const float* restrict X,
                          const float* restrict W,
                          const float* restrict b,
                          float* restrict Y,
                          int B, int I, int O, bool apply_relu) {
    #pragma acc kernels present (X[0:B*I], W[0:O*I], b[0:O], Y[0:B*O])
    { // Notice the 'present' keyword: this function will be called inside
      // a structured data region
      #pragma acc loop independent
      for (int i = 0; i < B; ++i) {
          for (int j = 0; j < O; ++j) {
              float sum = b[j];
              for (int k = 0; k < I; ++k) {
                  sum += X[i * I + k] * W[j * I + k];
              }
              if (apply_relu) {
                  Y[i * O + j] = sum > 0.0f ? sum : 0.0f;
              } else {
                  Y[i * O + j] = sum;
              }
          }
      }
    }
}
```

main

```
std::chrono::duration<double> elapsed_l1;
std::chrono::duration<double> elapsed_l2;
std::chrono::duration<double> elapsed_l3;

auto start_time = std::chrono::high_resolution_clock::now();
// Structured data region
#pragma acc data copyin(ptr_X[0:B*I1], \
                        ptr_W1[0:O1*I1], ptr_b1[0:O1], \
                        ptr_W2[0:O2*I2], ptr_b2[0:O2], \
                        ptr_W3[0:O3*I3], ptr_b3[0:O3]) \
                        create(ptr_H1[0:B*O1], ptr_H2[0:B*O2]) \
                        copyout(ptr_Logits[0:B*O3])
{
    // Layer 1
    auto start_l1 = std::chrono::high_resolution_clock::now();
    linear_layer_v4_step4(ptr_X, ptr_W1, ptr_b1, ptr_H1, B, I1, O1, true);
    auto end_l1 = std::chrono::high_resolution_clock::now();
    elapsed_l1 = end_l1 - start_l1;

    // Layer 2
    //...
}
auto end_time = std::chrono::high_resolution_clock::now();
```


OpenACC – versions

‘oacc_v4’: standard ‘parallel loop’ version

```
// V4: using the standard OpenACC approach: '#pragma acc parallel loop'
void linear_layer_v4_step4(const float* restrict X,
                          const float* restrict W,
                          const float* restrict b,
                          float* restrict Y,
                          int B, int I, int O, bool apply_relu) {

    #pragma acc parallel loop gang present(X[0:B*I], W[0:O*I], b[0:O], Y[0:B*O])
    for (int i = 0; i < B; ++i) {

        #pragma acc loop worker
        for (int j = 0; j < O; ++j) {
            float sum = b[j];

            #pragma acc loop vector reduction(+:sum)
            for (int k = 0; k < I; ++k) {
                sum += X[i * I + k] * W[j * I + k];
            }

            if (apply_relu) {
                Y[i * O + j] = sum > 0.0f ? sum : 0.0f;
            } else {
                Y[i * O + j] = sum;
            }
        }
    }
}
```

```
42, Generating present(W[:I*O],Y[:B*O],b[:O],X[:I*B])
Generating implicit firstprivate(B,I,O)
Generating NVIDIA GPU code
45, #pragma acc loop gang /* blockIdx.x */
48, #pragma acc loop worker(4) /* threadIdx.y */
52, #pragma acc loop vector(32) /* threadIdx.x */
Generating reduction(+:sum)
54, Vector barrier inserted for vector loop reduction
48, Loop is parallelizable
Generating implicit firstprivate(apply_relu)
52, Loop is parallelizable
36, Generating copyin(ptr_W3[:1280]) [if not already present]
Generating copyout(ptr_Logits[:1000]) [if not already present]
Generating copyin(ptr_X[:78400],ptr_W1[:200704],ptr_b1[:256],p
```

- Max warps per scheduler: 8
 - Schedulers per SM: 4
- ⇒ 32 warps per SM to fully exploit context switching
- If B is too low, SMs will be sitting mostly idle:
e.g. B=32 -> 2 blocks per SM & 4 warps per block -> 8/32 warps per scheduler
 - So V3 might actually run faster under certain conditions

CUDA naive

‘naive_v1’: the simplest fully functional CUDA version.

```
// =====  
// DEVICE MEMORY ALLOCATION (GPU)  
// =====  
float *d_X, *d_W1, *d_b1, *d_W2, *d_b2, *d_W3, *d_b3;  
float *d_H1, *d_H2, *d_Y;  
  
checkCuda(cudaMalloc((void**)&d_X, B * I1 * sizeof(float)));  
checkCuda(cudaMalloc((void**)&d_W1, O1 * I1 * sizeof(float)));  
checkCuda(cudaMalloc((void**)&d_b1, O1 * sizeof(float)));  
checkCuda(cudaMalloc((void**)&d_H1, B * O1 * sizeof(float)));  
  
// ...  
  
// ...  
  
// =====  
// TIMERS SETUP  
// =====  
cudaEvent_t start_h2d, stop_h2d;  
cudaEvent_t start_kernel, stop_kernel;  
cudaEvent_t start_d2h, stop_d2h;  
  
cudaEventCreate(&start_h2d); cudaEventCreate(&stop_h2d);  
cudaEventCreate(&start_kernel); cudaEventCreate(&stop_kernel);  
cudaEventCreate(&start_d2h); cudaEventCreate(&stop_d2h);
```

```
// =====  
// HOST-TO-DEVICE TRANSFER  
// =====  
cudaEventRecord(start_h2d);  
  
checkCuda(cudaMemcpy(d_X, h_X.data(), B * I1 * sizeof(float), cudaMemcpyHostToDevice));  
checkCuda(cudaMemcpy(d_W1, h_W1.data(), O1 * I1 * sizeof(float), cudaMemcpyHostToDevice));  
checkCuda(cudaMemcpy(d_b1, h_b1.data(), O1 * sizeof(float), cudaMemcpyHostToDevice));  
//...  
cudaEventRecord(stop_h2d);  
cudaEventSynchronize(stop_h2d);  
  
// =====  
// GRID AND BLOCK CONFIGURATION  
// =====  
dim3 blockSize(16, 16);  
  
// Grid calculation for each layer  
dim3 gridSize1((O1 + blockSize.x - 1) / blockSize.x, (B + blockSize.y - 1) / blockSize.y);  
dim3 gridSize2((O2 + blockSize.x - 1) / blockSize.x, (B + blockSize.y - 1) / blockSize.y);  
dim3 gridSize3((O3 + blockSize.x - 1) / blockSize.x, (B + blockSize.y - 1) / blockSize.y);
```

Block size 16x16 = 256 threads per block, because:

- 256 is a multiple of 32 (warp size) --> attempt to avoid divergence or inactive threads in the warp;
- it is low enough to allow allocating multiple blocks on a single SM --> first attempt to maximize occupancy and hide memory latency.

CUDA naive

```
// V1: Kernel naive
__global__ void linear_forward_naive(
    const float* X, const float* W, const float* b,
    float* Y, int B, int I, int O, bool apply_relu){

    // Calculation of global thread indices
    int row = blockIdx.y * blockDim.y + threadIdx.y; // batch index
    int col = blockIdx.x * blockDim.x + threadIdx.x; // output neuron index

    // Prevent out-of-bounds threads from reading/writing unallocated memory
    if (row < B && col < O) {
        float sum = b[col]; // accumulator initialized with the bias
        for (int k = 0; k < I; ++k) { // Dot product
            sum += X[row * I + k] * W[col * I + k];
        }

        // Apply ReLU directly in the thread registers.
        if (apply_relu && sum < 0.0f) {
            sum = 0.0f;
        }

        Y[row * O + col] = sum; // Single write to global memory
    }
}
```

Thread Mapping:

- Y axis (row) -> batch size (B)
- X axis (col) -> output neurons (O)

```
| main
| // =====
| // KERNEL EXECUTION (FORWARD PASS)
| // =====
| cudaEventRecord(start_kernel);
|
| // Layer 1
| linear_forward_naive<<<gridSize1, blockSize>>>(d_X, d_W1, d_b1, d_H1, B, I1, O1, true);
| checkCuda(cudaGetLastError());
|
| // Layer 2 ...
| // Layer 3 ...
|
| cudaEventRecord(stop_kernel);
| cudaEventSynchronize(stop_kernel);
|
| // =====
| // DEVICE-TO-HOST TRANSFER
| // =====
| cudaEventRecord(start_d2h);
|
| checkCuda(cudaMemcpy(h_Y.data(), d_Y, B * O3 * sizeof(float), cudaMemcpyDeviceToHost));
|
| cudaEventRecord(stop_d2h);
| cudaEventSynchronize(stop_d2h);
```

- Initializing the accumulator directly with the bias saves a memory access.
- ReLU on the fly avoids writing to global memory and then re-reading in a separate kernel.

CUDA naive – results

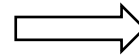
Using a fixed block dim3(16, 16) generates scheduling inefficiencies, and opposite ones depending on the layer size: tail effects on large layers and severe underutilization/divergence on small layers.

metric	L1	L2	L3
No Eligible Warps	96.32%	97.25%	92.96%
Achieved Occupancy	75.51%	52.30%	23.01%
Waves Per SM	1.75	0.88	0.11
Avg. Active Threads Per Warp	32	32	20.26

«the final partial wave of 48 blocks accounts for 50% of the runtime»

warp divergence

under-saturation: only 7 blocks for 16 SMs



To address the scheduling symptoms:

- enforce block_x = 32 (warp alignment),
- maximize threads per block,
- force spreading of blocks for Layer 3.

```
void compute_launch_config(int B, int O, dim3& blockSize, dim3& gridSize) {  
    const int NUM_SM = 16; // Number of SMs on my GTX 1650 Max-Q  
  
    int block_x = 32; // Warp size  
    int block_y = 32; // max threads  
  
    int grid_x = (O + block_x - 1) / block_x;  
    int grid_y = (B + block_y - 1) / block_y;  
    int total_blocks = grid_x * grid_y;  
  
    // Edge case (SM underutilization)  
    if (total_blocks < NUM_SM) {  
        block_y = B / NUM_SM;  
        if (block_y == 0) block_y = 1; // Safety check for very small batches  
    }  
  
    blockSize = dim3(block_x, block_y);  
    gridSize = dim3((O + blockSize.x - 1) / blockSize.x,  
                    (B + blockSize.y - 1) / blockSize.y);  
}
```

It won't fix the current poor memory handling though:

- **Bandwidth Waste:** used only 4.0 bytes out of 32 per transmitted sector
- **Excess Sectors :** 67% (L1,2) | 58% (L3)
- **Memory Stalls (LG Queue):** 154.8 cycles (L1) | 138.0 cycles (L2)
- **Hardware Saturation:** SM Busy at 3–5%.

(But it should facilitate the writing to Y, because of the warps «not folding» along the y direction)

CUDA naive – results

Metric	Layer	Naive Base	Naive Dynamic
Duration	L1	1.02 ms	2.33 ms
	L2	231 us	533 us
	L3	16.67 us	26.50 us
Achieved Occupancy	L1	75.51%	95.13%
	L2	52.30%	93.70%
	L3	23.01%	18.01%
Active Warps Per SM	L1	24.16	30.44
	L2	16.73	29.98
	L3	7.36	5.73
LG Queue	L1	154.8 cycles	366.0 cycles
	L2	138.0 cycles	419.2 cycles
	L3	13.6 cycles	18.2 cycles
Excessive Sectors	L1	67%	85%
	L2	67%	85%
	L3	58%	73%
Bandwidth waste	All	4.0 / 32 bytes	persistent
Scheduling Efficiency	L1	1.75	2.00
	L2	0.88	1.00
	L3	0.11	0.21
Active Threads per Warp	L3	20.26 / 32	10.43 / 32
Compute Throughput	All	SM Busy <10%	persistent
No Eligible Warps	All	>92%	persistent

The *scheduling* symptoms have been fixed, but the underlying *memory* condition worsened:

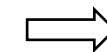
inserting 1024 threads per block (32 warps) without correcting the access pattern (4 bytes out of 32), many more warps simultaneously ask uncoalesced requests

→ (LG) queues saturate → stall cycles double → execution time doubles



Let's fix the memory access:

```
col = blockIdx.x * blockDim.x + threadIdx.x;
sum += X[row * I + k] * W[col * I + k];
```



k=0 -> W[0*I]

k=1 -> W[1*I]

...

→ to get the data elements of a full warp, 32 different requests to the VRAM, but extracting only a single float from each (<-> 4 bytes out of 32) .

This is the source of the observed saturation.

CUDA optimized

'opt_v1': solving the pathological memory access via tiling and the use of shared memory

```
//V1: Tiling 2D + shared memory
__global__ void linear_forward_tiled(const float* X, const float* W, const float* b,
                                     float* Y, int B, int I, int O, bool apply_relu){

    __shared__ float Xs[TILE_WIDTH][TILE_WIDTH]; // allocation of shared memory
    __shared__ float Ws[TILE_WIDTH][TILE_WIDTH];

    int tx = threadIdx.x; // Thread indices inside the tile
    int ty = threadIdx.y;

    // Global indices of the output matrix Y
    int row = blockIdx.y * TILE_WIDTH + ty; // B
    int col = blockIdx.x * TILE_WIDTH + tx; // O

    float sum = 0.0f; //stored in registers

    // Number of tiles needed to cover the inner dimension (I)
    int num_tiles = (I + TILE_WIDTH - 1) / TILE_WIDTH;

    for (int m = 0; m < num_tiles; ++m) { // Loop over tiles
```

```
// LOADING INTO SHARED MEMORY:
// Load tile of X: thread (ty, tx) loads element X[row, m*TILE + tx]
if (row < B && (m * TILE_WIDTH + tx) < I) {
    Xs[ty][tx] = X[row * I + (m * TILE_WIDTH + tx)];
} else {
    Xs[ty][tx] = 0.0f; // zero padding for out-of-bounds threads
}

// Load tile of W:
// W has shape [O, I] --> for coalesced reads from Global Memory,
// tx is mapped to the I-axis (contiguous in memory) and ty to the O-axis
int w_row = blockIdx.x * TILE_WIDTH + ty;
int w_col = m * TILE_WIDTH + tx;
if (w_row < O && w_col < I) {
    Ws[ty][tx] = W[w_row * I + w_col];
} else {
    Ws[ty][tx] = 0.0f; // padding for Layer 3 (because O = 10)
}

__syncthreads(); // wait for all threads to finish loading the tile

// PARTIAL DOT PRODUCT:
for (int k = 0; k < TILE_WIDTH; ++k) {
    // Xs[ty][k] -> element from row 'ty' of X
    // Ws[tx][k] -> element from row 'tx' of W (->column 'tx' of W^T)
    sum += Xs[ty][k] * Ws[tx][k];
}

__syncthreads(); //ensure all threads finish computing
} // End of loop

// WRITING TO GLOBAL MEMORY:
if (row < B && col < O) { // a valid element of Y is needed
    sum += b[col]; // add bias
    if (apply_relu && sum < 0.0f) {
        sum = 0.0f; // Kernel fusion: ReLU applied on-the-fly
    }
    Y[row * O + col] = sum;
}
}
```


CUDA optimized

Metric	Layer	Naive Dynamic	Tiling v1
Execution Time (us)	L1	2330	423.33
	L2	533	76.06
	L3	26.5	12.06
Excessive Sectors (%)	L1	85	0
	L2	85	0
	L3	73	0
LG queue (cycles)	L1	366	0
	L2	419.2	0
	L3	18.2	0
MIO Stalls (cycles)	L1	N/A	51.4
	L2	N/A	39.1
	L3	N/A	N/A
Bank Conflicts (way)	L1	0	9
	L2	0	9
	L3	0	9
IPC	L1	0.08	0.31
	L2	0.07	0.31
	L3	0.17	0.3
Compute Throughput (%)	L1	6.61	8.01
	L2	4.75	7.22
	L3	11.99	3.06
L1/TEX Hit Rate (%)	L1	99	14.34
	L2	99	28.51
	L3	99	6.06

→ nomore "uncoalesced global accesses"

→ different («better») stalls

→ low

→ low

→ expected, we're using the shared memory

- Solved the illed bandwidth toward the VRAM, cutting execution time in half.
- However the vertical rectangle shows other pathological behaviors;
- The horizontal one provides the answer: severe congestion in the physical banks of Shared Memory has been triggered (Bank Conflicts).
- Reading the tile columns, cause multiple collisions (9.0-way) → high MIO stalls → low IPC

CUDA optimized

‘opt_v2’: solves the Bank Conflicts

```
__shared__ float Xs[TILE_WIDTH][TILE_WIDTH];  
__shared__ float Ws[TILE_WIDTH][TILE_WIDTH + 1];
```

This is the only change made to ‘opt_v1’.

When the threads of a warp read

‘Ws[same column, different rows]’, they will now access different physical memory banks, lowering latency.

Metric	opt_v1	opt_v2
Execution Time (L1)	423.33 us	181.57 us
Bank Conflicts (Lettura)	9.0-way	0
MIO Stalls	51.4 cycles	14.4 cycles
Compute Throughput	8.01%	71.17%
IPC	0.31	0.90
Exec. Instructions (L1)	2.100.864	2.630.400
Bank Conflicts (Writing)	0	1.5-way
Occupancy	86.21%	83.74%

SM Busy = 25%
→ ALUs could
compute much more

→ to compute the new offsets
→ on the upload from
global to shared memo

INF Compute and Memory are well-balanced:

→ from the *ncu* report

Testing the effects of **B**:

Metric	B=100	B=8192	B=1
Waves Per SM (L1)	1.75	128	0.25
Achieved Occupancy	83.74%	99.56%	24.64%
L1/TEX Cache Throughput	90.52%	95.68%	45.70%
Compute Throughput	71.17%	75.54%	36.49%
SM Busy	25.73%	27.33%	13.20%

For high B no architectural degradation occurred, the metrics simply scaled.

- The hardware was flooded with work, making the tail effects statistically irrelevant.
- The kernel remains Memory Bound: the ALUs cannot compute any faster (L1 Cache/Shared Memory bus is at 95.68%).

Reducing B to 1 the shared memory loses its purpose, arithmetic intensity collapses, and the 2D block geometry causes massive thread waste → need to develop a dedicated kernel

CUDA vs OpenACC

‘**cuda_naive_v1** vs **oacc_v2**’: To evaluate the real cost of "compiler magic", through the "minimal effort" versions

Metric (L1)	cuda_naive_v1	oacc_v2	
Execution Time	1.02 ms	29.31 ms	1
Compute Throughput	15.15%	1.63%	
Memory Throughput	48.52%	1.63%	
DRAM Elapsed Cycles	14,166,016	406,339,584	2
Grid Over-provisioning	1 block per output	Grid: 25600, Block: 128	
Waves Per SM	2.15	200	3
IPC	0.14	0.06	
Stalls	154.8 cycles	403.2 cycles	
Global Reads Efficiency	4.0 of 32 bytes	31.9 of 32 bytes	4
Global Writes Efficiency	32 of 32 bytes	4.0 of 32 bytes	

- 1 30x slowdown
- 2 lethal implicit traffick
- 3 inefficient but not the main issue
- 4 automatically solved the memory stride

‘**cuda_naive_v2** vs **oacc_v4**’: To evaluate the advantages of "compiler magic" over non-optimal CUDA

Metric (L1)	cuda_naive_v2	oacc_v4
Execution Time	2.33 ms	402.24 us
IPC	0.08	2.15
Compute Throughput	6.61%	61.05%
Memory Stalls	366.0 cycles	4.5 cycles
Mapping	Dynamic Grid	Grid: 100, Block: 128
Global Reads Efficiency	4.0 of 32 bytes	31.9 of 32 bytes
DRAM Elapsed Cycles	32,534,528	5,525,504 cycles
Waves Per SM	2	0.78
Global Writes Efficiency	/	4.0 of 32 bytes

→ Providnig explicit data regions and correct hierarchical mapping, the OpenACC compiler generates code vastly superior to a naive CUDA kernel